

MPBS Lab

8085 Microprocessor Cheat Sheet

Quick-Reference for Microprocessor-Based Systems Lab

Organized so the instructions you'll reach for constantly are at the top.

Every instruction below shows its **actual hex opcode for every register variant** — no bit-pattern decoding required. Use Ctrl+F to jump to any mnemonic or hex byte.

1. Registers, Flags & Addressing (know this cold)

Internal Registers

Register	Size	Notes
A (Accumulator)	8-bit	All arithmetic/logic results land here
B, C, D, E, H, L	8-bit each	General purpose; pairable as BC, DE, HL
HL	16-bit pair	Almost always used as the memory pointer ("M")
SP (Stack Pointer)	16-bit	Points to top of stack
PC (Program Counter)	16-bit	Address of next instruction

Flag Register (Flag Word)

Bit	D7	D6	D5	D4	D3	D2	D1	D0
Flag	S	Z	X	AC	X	P	X	CY

- **S** (Sign) – set if result is negative (MSB = 1)
- **Z** (Zero) – set if result = 0
- **AC** (Auxiliary Carry) – carry out of bit 3 (used by **DAA**)
- **P** (Parity) – set if result has even number of 1-bits
- **CY** (Carry) – carry/borrow out of bit 7
- X = undefined bit

Addressing Modes

Mode	Example	Meaning
Immediate	<code>MVI A, 05H</code>	Data is in the instruction itself
Register	<code>MOV A, B</code>	Operand is in a register
Register Indirect	<code>MOV A, M</code>	Operand address is held in a register pair (usually HL)
Direct	<code>LDA 2050H</code>	Address is given directly in the instruction

2. TIER 1 — Instructions You'll Use in Almost Every Program

MOV r1, r2 (Move Register) — full hex grid

dest \ src	A	B	C	D	E	H	L	M
A	7F	78	79	7A	7B	7C	7D	7E
B	47	40	41	42	43	44	45	46
C	4F	48	49	4A	4B	4C	4D	4E
D	57	50	51	52	53	54	55	56
E	5F	58	59	5A	5B	5C	5D	5E
H	67	60	61	62	63	64	65	66

dest \ src	A	B	C	D	E	H	L	M
L	6F	68	69	6A	6B	6C	6D	6E
M	77	70	71	72	73	74	75	—

Row = destination register, column = source register. Read `MOV A, B` as row A, column B → **78**. (M,M doesn't exist — that slot is `HLT` = 76.)
Cycles/States: 1/4 (register-to-register) · 2/7 (any row or column involving M) · Flags: none

MVI r, data (Move Immediate) — opcode + data byte follows

r	A	B	C	D	E	H	L	M
Opcode	3E	06	0E	16	1E	26	2E	36

Bytes: 2 · Cycles/States: 2/7 (reg), 3/10 (M) · Flags: none — e.g. `MVI A, 05H` = **3E 05**

LXI rp, data16 (Load Register Pair Immediate) — opcode + low byte + high byte follow

rp	B	D	H	SP
Opcode	01	11	21	31

Bytes: 3 · Cycles/States: 3/10 · Flags: none — e.g. `LXI H, 2050H` = **21 50 20**

Jump / Call / Return / Stop

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
<code>JMP addr</code>	C3 + lo + hi	3	3 / 10	none
<code>J-condition addr</code>	see §6	3	2-3 / 7-10	none

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
CALL addr	CD + lo + hi	3	5 / 18	none
RET	C9	1	3 / 10	none
HLT	76	1	1+ / 5	none
NOP	00	1	1 / 4	none

3. TIER 2 — Arithmetic, Data Movement & I/O (used constantly)

ADD r (Add Register)

r	A	B	C	D	E	H	L	M
Hex	87	80	81	82	83	84	85	86

Cycles/States: 1/4 (reg), 2/7 (M) · Flags: Z,S,P,CY,AC

SUB r (Subtract Register)

r	A	B	C	D	E	H	L	M
Hex	97	90	91	92	93	94	95	96

Cycles/States: 1/4 (reg), 2/7 (M) · Flags: Z,S,P,CY,AC

INR r (Increment Register) — flags: Z,S,P,AC (**not CY**)

r	A	B	C	D	E	H	L	M
Hex	3C	04	0C	14	1C	24	2C	34

Cycles/States: 1/4 (reg), 3/10 (M)

DCR r (Decrement Register) — flags: Z,S,P,AC (**not CY**)

r	A	B	C	D	E	H	L	M
Hex	3D	05	0D	15	1D	25	2D	35

Cycles/States: 1/4 (reg), 3/10 (M)

INX rp / DCX rp (Increment/Decrement Register Pair) — no flags affected

rp	B	D	H	SP
INX Hex	03	13	23	33
DCX Hex	0B	1B	2B	3B

Cycles/States: 1/6 (both)

CMP r (Compare Register) — A unchanged; sets Z if equal, CY if A < r

r	A	B	C	D	E	H	L	M
Hex	BF	B8	B9	BA	BB	BC	BD	BE

Cycles/States: 1/4 (reg), 2/7 (M) · Flags: Z,S,P,CY,AC

Direct Addressing / Immediate Compare / I/O

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
STA addr	32 + lo + hi	3	4 / 13	none
LDA addr	3A + lo + hi	3	4 / 13	none
CPI data	FE + data	2	2 / 7	Z,S,P,CY,AC
IN port	DB + port	2	3 / 10	none
OUT port	D3 + port	2	3 / 10	none
JNZ addr	C2 + lo + hi	3	2-3 / 7-10	none
JZ addr	CA + lo + hi	3	2-3 / 7-10	none

Gotcha: `INR` / `DCR` affect all flags **except** `CY` — a classic lab trap when using them as loop counters and testing for a borrow.

4. TIER 3 — Logic, Rotates & Stack (common)

ANA r (Logical AND with Accumulator)

r	A	B	C	D	E	H	L	M
Hex	A7	A0	A1	A2	A3	A4	A5	A6

Cycles/States: 1/4 (reg), 2/7 (M) · Flags: Z,S,P,AC,CY(cleared)

ORA r (Logical OR with Accumulator)

r	A	B	C	D	E	H	L	M
Hex	B7	B0	B1	B2	B3	B4	B5	B6

Cycles/States: 1/4 (reg), 2/7 (M) · Flags: Z,S,P,CY,AC(cleared)

XRA r (Logical XOR with Accumulator) — `XRA A` (hex **AF) is the classic "clear A and CY in one shot" trick**

r	A	B	C	D	E	H	L	M
Hex	AF	A8	A9	AA	AB	AC	AD	AE

Cycles/States: 1/4 (reg), 2/7 (M) · Flags: Z,S,P,CY,AC(cleared)

Logic / Rotate / Complement (immediate & single-byte)

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
<code>ANI data</code>	E6 + data	2	2 / 7	Z,S,P,CY,AC
<code>ORI data</code>	F6 + data	2	2 / 7	Z,S,P,CY,AC

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
XRI data	EE + data	2	2 / 7	Z,S,P,CY,AC
RLC	07	1	1 / 4	CY only
RRC	0F	1	1 / 4	CY only
RAL	17	1	1 / 4	CY only
RAR	1F	1	1 / 4	CY only
CMA	2F	1	1 / 4	none

PUSH rp / POP rp (Stack) — note: PSW takes the SP's opcode slot, SP itself is invalid here

rp	B	D	H	PSW
PUSH Hex	C5	D5	E5	F5
POP Hex	C1	D1	E1	F1

Cycles/States: PUSH 3/12 · POP 3/10 · Flags: none (Z,S,P,CY,AC restored only on POP PSW)

DAD rp (Add Register Pair to HL) — only CY affected

rp	B	D	H	SP
Hex	09	19	29	39

Cycles/States: 3 / 10

XCHG (Exchange HL with DE)

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
XCHG	EB	1	1 / 4	none

5. TIER 4 — Everything Else (less frequent but you'll need them)

Immediate Arithmetic

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
ADI data	C6 + data	2	2 / 7	Z,S,P,CY,AC
SUI data	D6 + data	2	2 / 7	Z,S,P,CY,AC
ACI data	CE + data	2	2 / 7	Z,S,P,CY,AC
SBI data	DE + data	2	2 / 7	Z,S,P,CY,AC

ADC r (Add Register with Carry)

r	A	B	C	D	E	H	L	M
Hex	8F	88	89	8A	8B	8C	8D	8E

Cycles/States: 1/4 (reg), 2/7 (M) · Flags: Z,S,P,CY,AC

SBB r (Subtract Register with Borrow)

r	A	B	C	D	E	H	L	M
Hex	9F	98	99	9A	9B	9C	9D	9E

Cycles/States: 1/4 (reg), 2/7 (M) · Flags: Z,S,P,CY,AC

LDAX rp / STAX rp — **only rp = B or D is legal** (not H, not SP)

rp	B	D
LDAX Hex	0A	1A
STAX Hex	02	12

Cycles/States: 2 / 7 · Flags: none

Direct HL Load/Store & Indirect Jumps

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
LHLD addr	2A + lo + hi	3	5 / 16	none
SHLD addr	22 + lo + hi	3	5 / 16	none
XTHL	E3	1	5 / 16-18	none
SPHL	F9	1	1 / 6	none
PCHL	E9	1	1 / 6	none

Conditional Call / Return — see full condition table in §6 for all 8 hex codes each

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
C-condition addr	see §6	3	2-5 / 9-18	none
R-condition	see §6	1	1-3 / 6-12	none

RST n (Restart)

n	0	1	2	3	4	5	6	7
Hex	C7	CF	D7	DF	E7	EF	F7	FF

Cycles/States: 3 / 12 · Flags: none (see §6 for the restart *address* each one jumps to)

Flags / Interrupts / 8085-only

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
DAA	27	1	1 / 4	all
CMC	3F	1	1 / 4	CY
STC	37	1	1 / 4	CY
EI	FB	1	1 / 4	none

Mnemonic	Hex Opcode	Bytes	Cycles/States	Flags
DI	F3	1	1 / 4	none
RIM (8085 only)	20	1	1 / 4	none
SIM (8085 only)	30	1	1 / 4	none

6. Conditional Branch Reference — every hex code for every condition

Condition	Meaning	CCC	JMP hex	CALL hex	RET hex
NZ	Z = 0 (not zero)	000	C2	C4	C0
Z	Z = 1 (zero)	001	CA	CC	C8
NC	CY = 0 (no carry)	010	D2	D4	D0
C	CY = 1 (carry)	011	DA	DC	D8
PO	P = 0 (parity odd)	100	E2	E4	E0
PE	P = 1 (parity even)	101	EA	EC	E8
P	S = 0 (plus/positive)	110	F2	F4	F0
M	S = 1 (minus/negative)	111	FA	FC	F8

Restart (RST) address table:

RST	Hex	Address	RST	Hex	Address
0	C7	0000H	4	E7	0020H
1	CF	0008H	5	EF	0028H
2	D7	0010H	6	F7	0030H
3	DF	0018H	7	FF	0038H

7. Full Opcode Table (flat A-Z reference, hex, by mnemonic group)

Data Transfer

Instr	Hex	Instr	Hex	Instr	Hex	Instr	Hex
MOV A,A	7F	MOV B,A	47	MOV C,A	4F	MOV D,A	57
MOV A,B	78	MOV B,B	40	MOV C,B	48	MOV D,B	50
MOV A,C	79	MOV B,C	41	MOV C,C	49	MOV D,C	51
MOV A,D	7A	MOV B,D	42	MOV C,D	4A	MOV D,D	52
MOV A,E	7B	MOV B,E	43	MOV C,E	4B	MOV D,E	53
MOV A,H	7C	MOV B,H	44	MOV C,H	4C	MOV D,H	54
MOV A,L	7D	MOV B,L	45	MOV C,L	4D	MOV D,L	55
MOV A,M	7E	MOV B,M	46	MOV C,M	4E	MOV D,M	56
MOV E,A	5F	MOV H,A	67	MOV L,A	6F	MOV M,A	77
MOV E,B	58	MOV H,B	60	MOV L,B	68	MOV M,B	70
MOV E,C	59	MOV H,C	61	MOV L,C	69	MOV M,C	71
MOV E,D	5A	MOV H,D	62	MOV L,D	6A	MOV M,D	72
MOV E,E	5B	MOV H,E	63	MOV L,E	6B	MOV M,E	73
MOV E,H	5C	MOV H,H	64	MOV L,H	6C	MOV M,H	74

Instr	Hex	Instr	Hex	Instr	Hex	Instr	Hex
MOV E,L	5D	MOV H,L	65	MOV L,L	6D	MOV M,L	75
MOV E,M	5E	MOV H,M	66	MOV L,M	6E	HLT	76

Instr	Hex	Instr	Hex	Instr	Hex
MVI A,byte	3E	MVI E,byte	1E	LXI B,d16	01
MVI B,byte	06	MVI H,byte	26	LXI D,d16	11
MVI C,byte	0E	MVI L,byte	2E	LXI H,d16	21
MVI D,byte	16	MVI M,byte	36	LXI SP,d16	31
LDAX B	0A	STAX B	02	LHLD adr	2A
LDAX D	1A	STAX D	12	SHLD adr	22
LDA adr	3A	STA adr	32	XCHG	EB

Arithmetic & Logical

Instr	Hex	Instr	Hex	Instr	Hex	Instr	Hex
ADD B	80	ADC B	88	SUB B	90	SBB B	98
ADD C	81	ADC C	89	SUB C	91	SBB C	99
ADD D	82	ADC D	8A	SUB D	92	SBB D	9A
ADD E	83	ADC E	8B	SUB E	93	SBB E	9B
ADD H	84	ADC H	8C	SUB H	94	SBB H	9C
ADD L	85	ADC L	8D	SUB L	95	SBB L	9D
ADD M	86	ADC M	8E	SUB M	96	SBB M	9E
ADD A	87	ADC A	8F	SUB A	97	SBB A	9F

Instr	Hex	Instr	Hex	Instr	Hex	Instr	Hex
ANA B	A0	XRA B	A8	ORA B	B0	CMP B	B8
ANA C	A1	XRA C	A9	ORA C	B1	CMP C	B9
ANA D	A2	XRA D	AA	ORA D	B2	CMP D	BA

Instr	Hex	Instr	Hex	Instr	Hex	Instr	Hex
ANA E	A3	XRA E	AB	ORA E	B3	CMP E	BB
ANA H	A4	XRA H	AC	ORA H	B4	CMP H	BC
ANA L	A5	XRA L	AD	ORA L	B5	CMP L	BD
ANA M	A6	XRA M	AE	ORA M	B6	CMP M	BE
ANA A	A7	XRA A	AF	ORA A	B7	CMP A	BF

Instr	Hex	Instr	Hex	Instr	Hex	Instr	Hex
INR A	3C	DCR A	3D	INX B	03	DCX B	0B
INR B	04	DCR B	05	INX D	13	DCX D	1B
INR C	0C	DCR C	0D	INX H	23	DCX H	2B
INR D	14	DCR D	15	INX SP	33	DCX SP	3B
INR E	1C	DCR E	1D	DAD B	09	DAD D	19
INR H	24	DCR H	25	DAD H	29	DAD SP	39
INR L	2C	DCR L	2D				
INR M	34	DCR M	35				

Instr	Hex	Instr	Hex	Instr	Hex
ADI byte	C6	ANI byte	E6	DAA	27
ACI byte	CE	XRI byte	EE	CMA	2F
SUI byte	D6	ORI byte	F6	STC	37
SBI byte	DE	CPI byte	FE	CMC	3F
				RLC/RRC/RAL/RAR	07/0F/17/1F

Branch, Stack, I/O, Control

Instr	Hex	Instr	Hex	Instr	Hex
JMP adr	C3	CALL adr	CD	RET	C9
PUSH B/D/H/PSW	C5/D5/E5/F5	POP B/D/H/PSW	C1/D1/E1/F1	RST 0-7	C7,CF,D
IN byte	DB	OUT byte	D3	XTHL	E3
DI / EI	F3 / FB	NOP	00	SPHL	F9
RIM (8085)	20	SIM (8085)	30	PCHL	E9

8. Example Assembly Programs

Assumes program loaded starting at 2000H (a common lab convention — adjust to your monitor's actual load address). Data area starts at 2050H.

8.1 Conditional (If/Else) — Find the Larger of Two Numbers

Compares two bytes at 2050H/2051H and stores the larger one at 2052H.

Assembly	Address	Machine Code	Comments
LDA 2050H	2000	3A 50 20	A = first number
MOV B, A	2003	47	save it in B
LDA 2051H	2004	3A 51 20	A = second number
CMP B	2007	B8	compare A (2nd) with B (1st); sets CY if A<B
JNC 200CH	2008	D2 0C 20	if A ≥ B, A already holds the larger — skip ahead
MOV A, B	200B	78	else the first number (B) was larger

Assembly	Address	Machine Code	Comments
STA 2052H	200C	32 52 20	store the result
HLT	200F	76	stop

8.2 Loop — Sum of an Array of N Numbers

Adds 10 (0AH) bytes starting at 2051H; stores the 8-bit sum at 2080H.

Assembly	Address	Machine Code	Comments
LXI H, 2051H	2000	21 51 20	HL = pointer to first element
MVI B, 0AH	2003	06 0A	B = loop counter (10 elements)
MVI A, 00H	2005	3E 00	A = running sum, start at 0
ADD M	2007	86	loop: A = A + (HL)
INX H	2008	23	advance pointer to next element
DCR B	2009	05	decrement counter
JNZ 2007H	200A	C2 07 20	loop back until B = 0
STA 2080H	200D	32 80 20	store final sum
HLT	2010	76	stop

8.3 Array — Find the Largest Element

Scans 10 bytes starting at 2051H and stores the largest at 2080H.

Assembly	Address	Machine Code	Comments
LXI H, 2051H	2000	21 51 20	HL = pointer to array
MVI B, 09H	2003	06 09	counter = (N-1) remaining comparisons
MOV A, M	2005	7E	A = first element (assume largest so far)
INX H	2006	23	point to next element
CMP M	2007	BE	loop: compare A with (HL)
JNC 200CH	2008	D2 0C 20	if $A \geq (HL)$, current max stands — skip update
MOV A, M	200B	7E	else (HL) is the new largest
INX H	200C	23	advance pointer
DCR B	200D	05	decrement counter
JNZ 2007H	200E	C2 07 20	loop until all elements checked
STA 2080H	2011	32 80 20	store the largest value found
HLT	2014	76	stop

8.4 Nested Loop — Software Time Delay Subroutine

A classic double-loop delay (called with `CALL`); returns with `RET`.

Assembly	Address	Machine Code	Comments
MVI D, FFH	2000	16 FF	D = outer loop count
MVI E, FFH	2002	1E FF	outer: E = inner loop count
DCR E	2004	1D	inner: decrement E

Assembly	Address	Machine Code	Comments
JNZ 2004H	2005	C2 04 20	repeat inner loop until E = 0
DCR D	2008	15	decrement D
JNZ 2002H	2009	C2 02 20	repeat outer loop (reloads E) until D = 0
RET	200C	C9	return to caller

8.5 Array — Block Move (Copy an Array)

Copies 10 bytes from a source block at 2050H to a destination block at 2070H.

Assembly	Address	Machine Code	Comments
LXI H, 2050H	2000	21 50 20	HL = source pointer
LXI D, 2070H	2003	11 70 20	DE = destination pointer
MVI B, 0AH	2006	06 0A	B = count (10 bytes)
MOV A, M	2008	7E	loop: A = source byte
STAX D	2009	12	write A to destination (via DE)
INX H	200A	23	advance source pointer
INX D	200B	13	advance destination pointer
DCR B	200C	05	decrement counter
JNZ 2008H	200D	C2 08 20	loop until all bytes copied
HLT	2010	76	stop

9. Common Lab Gotchas

- **INR / DCR never affect CY.** If you're checking for a borrow/carry in a loop, use **DCX / INX** on a pair, or use **SUB / CMP** instead.
- **ANA / ORA / XRA always clear CY** (and AC behavior differs between ANA and the others) — don't rely on carry surviving a logic op.
- **Register pair `rp = SP` cannot be used with `PUSH / POP / LDAX / STAX`** — only B, D, H (and PSW for PUSH/POP) are valid there.
- **LDAX / STAX** only work with register pairs **B** or **D** — never H (use **`MOV r, M / MOV M, r`** for HL-based access instead).
- **Conditional jump/call/return cycle counts differ** depending on whether the condition is taken (see §2/§5) — matters for timing-critical delay routines.
- **8085 vs 8080 state counts differ** for several instructions (e.g., **`MOV r1, r2`** is 4 states on 8085 but 5 on 8080) — use the 8085 column for this lab.